

Bytecode without the Black Magic

Java's Class-File API in Practice

Confitura 2026, Petro Velychko

// TODO: Read → Generate → Transform

READ → GENERATE → TRANSFORM

Read

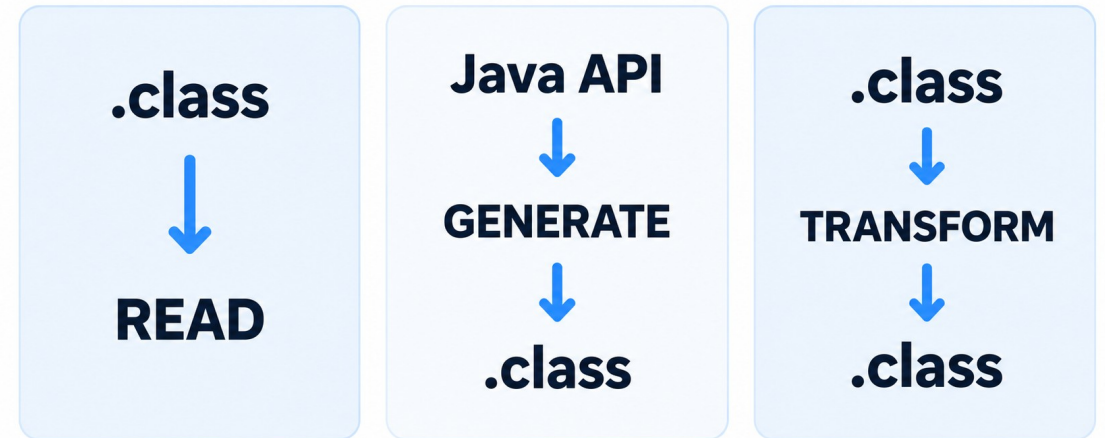
Understand an existing .class

Generate

Create a .class from Java code

Transform

Modify an existing .class



About me



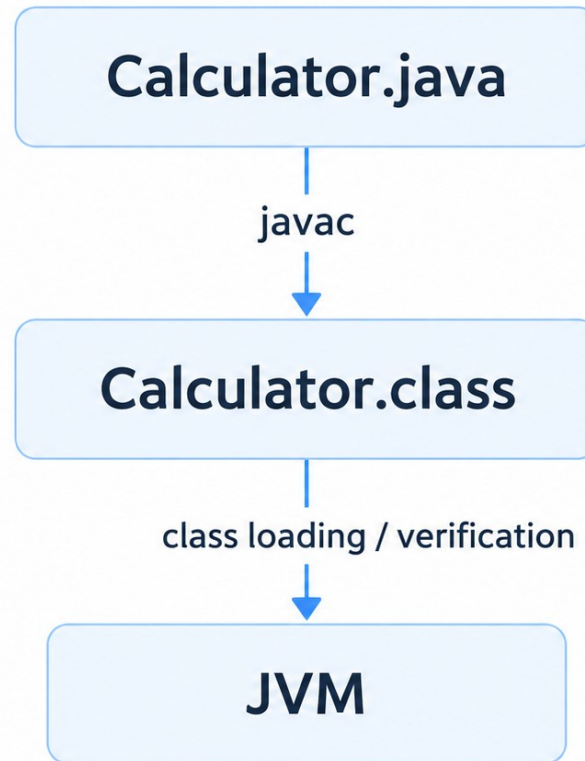
- Petro Velychko
- Software Developer
- Java, Kotlin, Scala
- LinkedIn : <https://www.linkedin.com/in/petervelychko/>

You Already Use Bytecode

You probably already use bytecode manipulation

- Frameworks
- Java agents
- Profilers / APM
- Code coverage
- Mocking
- Build tools
- Static analysis
- Language implementations

What Java Actually Runs



Anatomy of a .class

A class file contains

Class



For a method:

Method



magic = 0xCAFEBAE

Java 25 class-file major version = 69

From Java to Bytecode

```
public int add(int a, int b) {  
    return a + b;  
}
```

0: iload_1

1: iload_2

2: iadd

3: ireturn

JVM Operand Stack

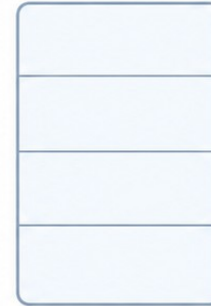
Local variables

0 → this

1 → a

2 → b

Operand stack



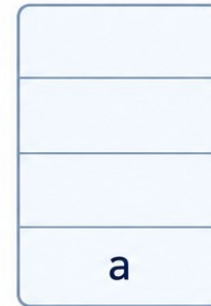
Instruction steps

JVM Operand Stack

Local variables

0 → this
1 → a
2 → b

Operand stack



Instruction steps

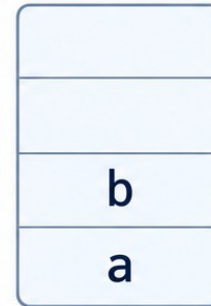
`iload_1` `[]` → `[a]`

JVM Operand Stack

Local variables

0 → this
1 → a
2 → b

Operand stack



Instruction steps

iload_1	[]	→	[a]
iload_2	[a]	→	[a, b]

JVM Operand Stack

Local variables

0 → this
1 → a
2 → b

Operand stack



Instruction steps

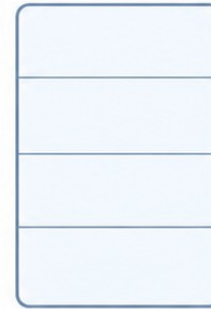
iload_1	[]	→	[a]
iload_2	[a]	→	[a, b]
iadd	[a, b]	→	[a + b]

JVM Operand Stack

Local variables

0 → this
1 → a
2 → b

Operand stack



Instruction steps

iload_1	[]	→	[a]
iload_2	[a]	→	[a, b]
iadd	[a, b]	→	[a + b]
ireturn	[a + b]	→	[]

Constant Pool

Class files use symbolic references

Constant pool:

#1 = Methodref	#2.#3	// java/lang/Object."<init>":()V
#2 = Class	#4	// java/lang/Object
#3 = NameAndType	#5:#6	// "<init>":()V
#4 = Utf8	java/lang/Object	
#5 = Utf8	<init>	
#6 = Utf8	()V	
...		
#13 = String	#14	// Hello
#14 = Utf8	Hello	
...		

Code:

...		
0: getstatic	#7	// Field java/lang/System.out:Ljava/io/PrintStream;
3: ldc	#13	// String Hello
5: invokevirtual	#15	// Method java/io/PrintStream.println:(Ljava/lang/String;)V
8: return		

Bytecode tooling is not new

ASM

Low-level class-file and bytecode manipulation

Byte Buddy

Higher-level code generation and instrumentation

Javassist

Source-like / structural bytecode manipulation

BCEL

Object-oriented class-file parsing, generation, and bytecode manipulation

The ecosystem already has excellent tools

java.lang.classfile

```
import java.lang.classfile.*;
```

Parse

```
ClassFile.of().parse(path)
```

Generate

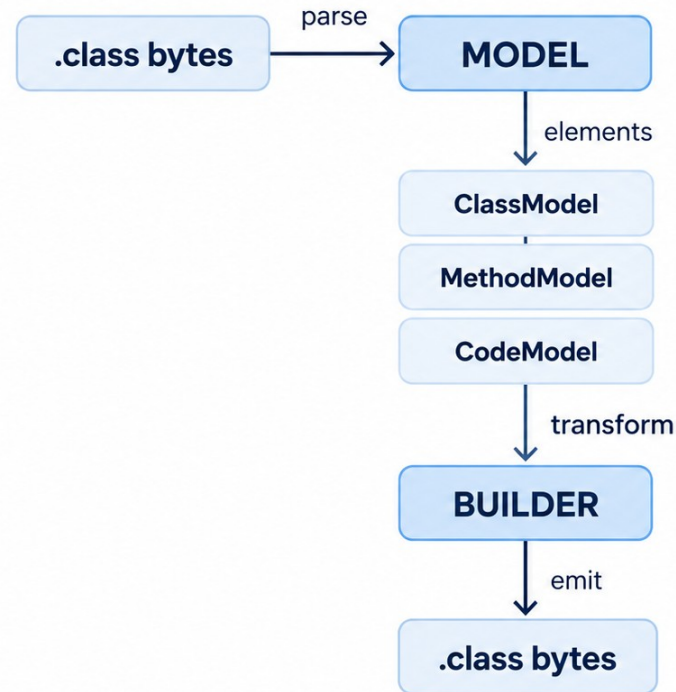
```
ClassFile.of().build(...)
```

Transform

```
ClassFile.of().transformClass(...)
```

Standard API since JDK 24

API Mental Model



Model = what exists

Builder = what you want to emit

Transform = how existing elements are preserved, changed, or dropped

Demo 1: READ

READ

What did javac actually produce?

`Calculator.class`

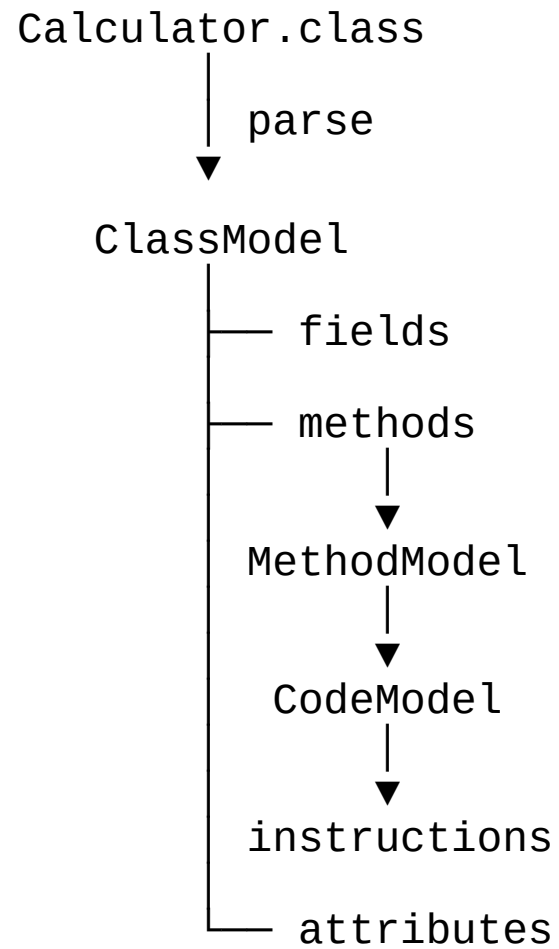


`ClassModel`

DEMO: Inspect Calculator.class

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public int answer() {  
        return 42;  
    }  
}
```

What We Just Did



Can We Skip javac?

Normal:

Generated.java



javac



Generated.class

Alternative:

Class-File API



Generated.class

Our Goal to Generate

```
public class Generated {  
    public static int answer() {  
        return 42;  
    }  
}
```

There is no Generated.java

Demo 2: GENERATE

GENERATE

`ClassFile.build(...)`



`ClassBuilder`



`MethodBuilder`



`CodeBuilder`



`Generated.class`

DEMO: Generate a Class

```
ClassFile.build(...)
```

```
ClassBuilder
```

```
CodeBuilder
```

```
loadConstant(42)
```

```
ireturn()
```

Generate: The Important Part

Describe intent, not file encoding

"I need a method"

↓

"I need some code"

↓

"load 42"

↓

"return int"

Instead of:

constant_pool_index = ?

attribute_length = ?

byte_offset = ?

New Requirement: 42 Is Wrong

Before:

```
public int answer() {  
    return 42;  
}
```

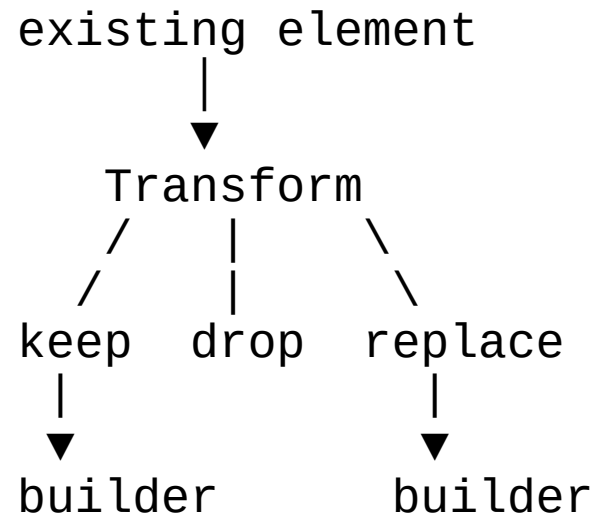
After:

```
public int answer() {  
    return 43;  
}
```

Change the class.

Don't recompile the source.

Transforms Are Element Pipelines



<code>builder.with(element)</code>	= preserve
<code>emit something else</code>	= replace
<code>emit nothing</code>	= drop

Demo 3: TRANSFORM

TRANSFORM

Calculator.class



ClassModel



transform



Calculator.class

42 → 43

DEMO: Transform Calculator

Target method:
answer()

Target instruction:
42

Replacement:
43

What Real Transformations Look Like

Our demo:

42
↓
43

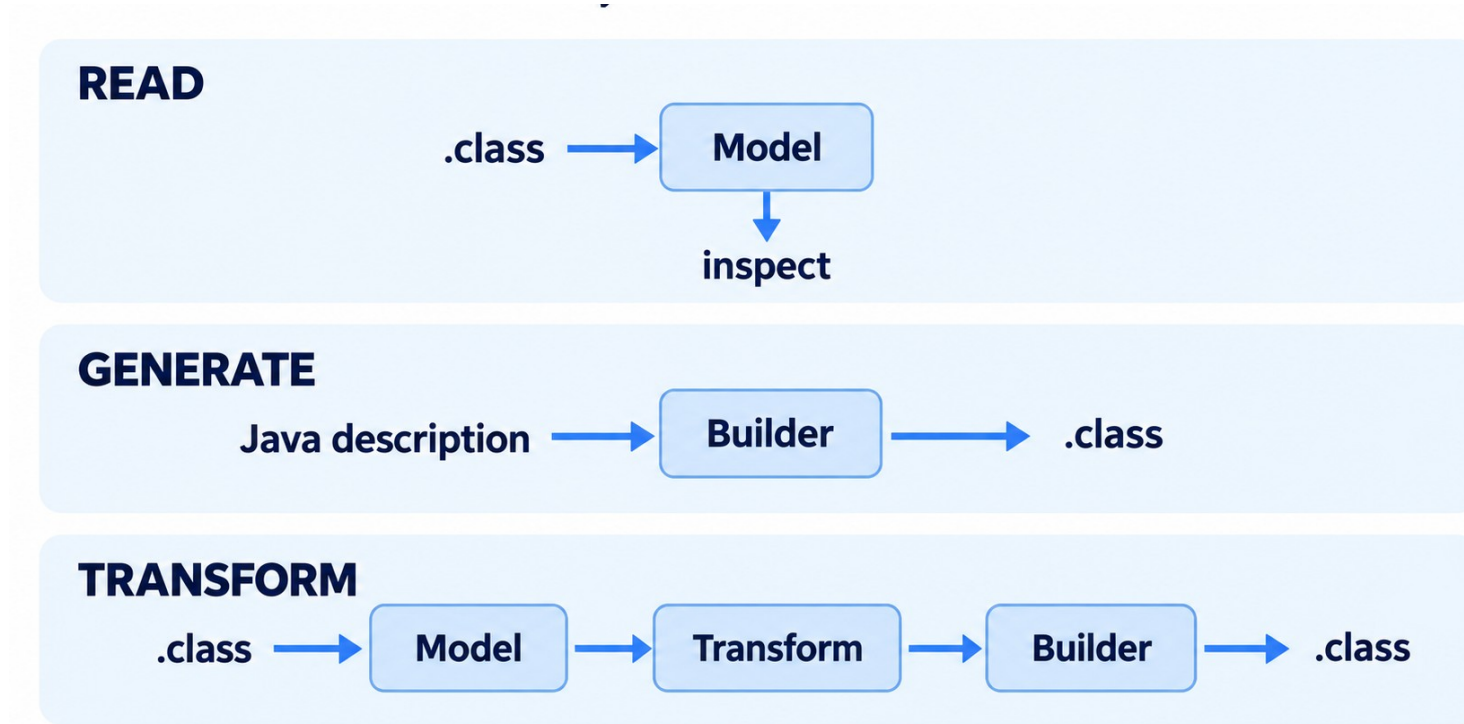
Real tooling:

method
↓
inject tracing
↓
method + tracing

method
↓
coverage transform
↓
method + counters

class
↓
instrumentation
↓
modified class

One API, Three Directions



Elements + Builders + Transforms

Class-File API vs ASM vs Byte Buddy

	Class-File API	ASM	Byte Buddy
Level	Class-file model	Low-level bytecode	Higher-level generation
In JDK	Yes	No	No
Parse	Yes	Yes	Yes
Generate	Yes	Yes	Yes
Transform	Yes	Yes	Yes
Bytecode knowledge	Useful / often needed	Typically needed	Often less
Ecosystem maturity	Newer	Very mature	Very mature

Different abstraction levels. Different trade-offs.

Less Magic != No Rules

The API removes accidental complexity

It does NOT remove JVM semantics

You still need to understand:

- Types
- Operand stack
- Control flow
- Descriptors
- Verification
- Stack-map frames

Bytecode Without the Black Magic

Bytecode is structured data

The JVM is understandable

Java now has a standard API for class files

READ

`ClassFile.of()`

Bytecode Without the Black Magic

Bytecode is structured data

The JVM is understandable

Java now has a standard API for class files

READ → GENERATE

`ClassFile.of()`

Bytecode Without the Black Magic

Bytecode is structured data

The JVM is understandable

Java now has a standard API for class files

READ → GENERATE → TRANSFORM

`ClassFile.of()`

Three Things to Remember

1. Bytecode is understandable
2. Models + Builders + Transforms
3. The API removes bookkeeping - not JVM semantics

Thank You

GitHub repository:

<https://github.com/petervelychko/bytecode-without-black-magic-demo>

